CYBER SECURITY

Practical Web Application Security Assessment

Assignment 1

Course	Cyber Security
Section	E4
Instructor	Hafiz Usama Ishtiaq
Submitted by	sharjeel

1. Introduction and Objectives

This report documents a practical review of Altoro Mutual, an intentionally vulnerable banking demo used for security education. The assessment focused on how the site handles login credentials and user-supplied search text. The aim was to observe possible SQL injection and cross-site scripting (XSS) indicators, explain their potential impact, and recommend practical fixes.

Testing was limited to the named training site and the simple, non-destructive checks described in the assignment. The findings below reflect the responses recorded in the supplied evidence; they are not a full penetration test of a production banking system.

2. Website Overview

Altoro Mutual is a demonstration banking website with informational pages and an online banking area. The reviewed sections and key features were:

- Personal Banking — information about services and products for individual customers.
- Small Business Banking — information about banking services for small businesses.
- Inside Altoro Mutual — background and general information about the demonstration bank.
- Online Banking Sign In — accepts a username and password for authentication.
- Search and Feedback — Search returns a response based on a query; Feedback lets visitors submit comments.

2.1 Location and Security Relevance

Function	Location recorded	Why it needs testing
Sign In	Online Banking; /login.jsp	Processes credentials and must resist authentication bypass and unsafe database queries.
Search	Top-right search area; /search.jsp?query=	User input may be reflected in a response, creating a possible XSS risk if output is not encoded.
Feedback	Feedback link; /feedback.jsp	Submitted text may be stored or displayed later and must not be interpreted as unsafe HTML or script.

3. Testing Methodology and Tools

The assessment used manual, black-box checks in a web browser. For SQL injection, an ordinary failed login was recorded as the baseline, then single apostrophes were tested separately in the username and password fields. An optional instructor-described input was also tested. For XSS, the Search field was selected because it displayed submitted query text; an ordinary query, an HTML-marker string, and the standard harmless alert test were compared. Responses were documented with screenshots.

Tools used: a web browser and the Altoro Mutual web interface. No automated scanner was reported in the supplied material. The tests were observational and limited to the authorized training environment.

4. Findings and Observations

4.1 SQL Injection — Online Banking Sign In

SQL injection occurs when user input changes the structure or meaning of a database query. A common cause is building SQL by concatenating input into query text instead of binding it as a parameter.

Test	Recorded application response	Interpretation
Baseline: username test; password test	“Login Failed: We're sorry, but this username or password was not found in our system. Please try again”	Ordinary failed login; baseline for comparison.
Username: apostrophe; password test	“Syntax error: Encountered "test" at line 1, column 62.”	Database/query syntax detail differs from the baseline.
Username test; password: apostrophe	“Syntax error: Encountered "\'" at line 1, column 66.”	Another SQL-related syntax error; unsafe input handling is indicated.
Username: <' OR '1'='1' --> Password test	Returned “Hello Admin User” and a welcome/pre-approved card message.	Apparent authentication bypass; strong evidence of SQL injection in this test context.

Classification and Potential Impact

Finding: Vulnerable to SQL injection in the tested login flow. The baseline failed normally, both single-quote probes produced syntax errors, and the optional test returned an authenticated-looking welcome page. These results strongly indicate that input can affect query interpretation. In a real banking application, similar weakness could permit authentication bypass or unauthorized access to customer data; depending on database permissions, records could also be changed or deleted.

Recommended Fixes

- Use parameterized queries/prepared statements for all database operations.

- Return generic login errors and keep SQL exceptions, stack traces, and query details out of user-facing responses.
- Apply least-privilege permissions to the application's database account.
- Validate expected input formats as an additional layer, not as a replacement for parameterization.

4.2 Cross-Site Scripting (XSS) — Search

Reflected XSS occurs when input is included unsafely in the immediate response. Stored XSS occurs when input is saved and later rendered unsafely to users. The Search field was selected because the query was shown in the response page.

Test	Recorded observation	Interpretation
Baseline: `test search`	"No results were found for the query: test search."	Ordinary search response.
HTML marker: `testing<b>bold</b>testing`	"System returned: testing**bold**testing."	Tags were not shown literally; this alone is not conclusive proof of script execution.
Script test: `<script>alert('XSS-Test')</script>`	A dialog box appeared on the screen.	JavaScript execution was observed; strong evidence of reflected XSS.

Classification and Potential Impact

Finding: Vulnerable to reflected XSS in the tested Search response. The alert dialog is direct evidence that the submitted script executed in the browser, while the input appeared in the immediate response. The evidence does not establish stored XSS. In a real banking site, XSS could be used for phishing, altering page content, or triggering actions within a victim's authenticated session, depending on the site's controls and the victim's interaction.

Recommended Fixes

- Apply context-appropriate output encoding before inserting user-controlled values into HTML.

- Use templates/frameworks that escape output by default and avoid unsafe HTML insertion.
- Deploy a restrictive Content Security Policy as an additional defense.
- Set sensitive session cookies to HttpOnly where appropriate; this reduces direct JavaScript access to cookies but does not fix XSS itself.

4.3 Security Analysis

SQL injection vs. XSS SQL injection targets the way an application builds database queries; XSS targets the way a browser interprets untrusted content in a page. SQL injection can affect database operations, while XSS runs script in the user's browser under the site's origin.

CIA triad and SQL injection Confidentiality may be affected through unauthorized data access; integrity through changing or deleting records; and availability if data or database operations are disrupted. The observed login behavior chiefly demonstrates a risk to confidentiality and authentication, not proof of every possible impact.

Why server-side authorization matters Client-side checks can be bypassed or modified by a user. The server must verify permissions for every protected request and resource, including whether the logged-in user is allowed to access a particular account or transaction.

How parameterized queries help They send the SQL structure and input values separately, so input is treated as data rather than being parsed as part of the SQL command.

How output encoding helps prevent XSS Encoding special characters for the context where they are displayed prevents text such as angle brackets from being interpreted as active HTML or script. The correct encoding depends on whether the value is placed in HTML, an attribute, a URL, or JavaScript.

5. Screenshot Evidence and Captions

All 14 screenshots embedded in the supplied assignment document have been retained below in their original image form and given captions. The captions describe the visible page or recorded test; they do not add results beyond the supplied evidence.

SCREENSHOT 01

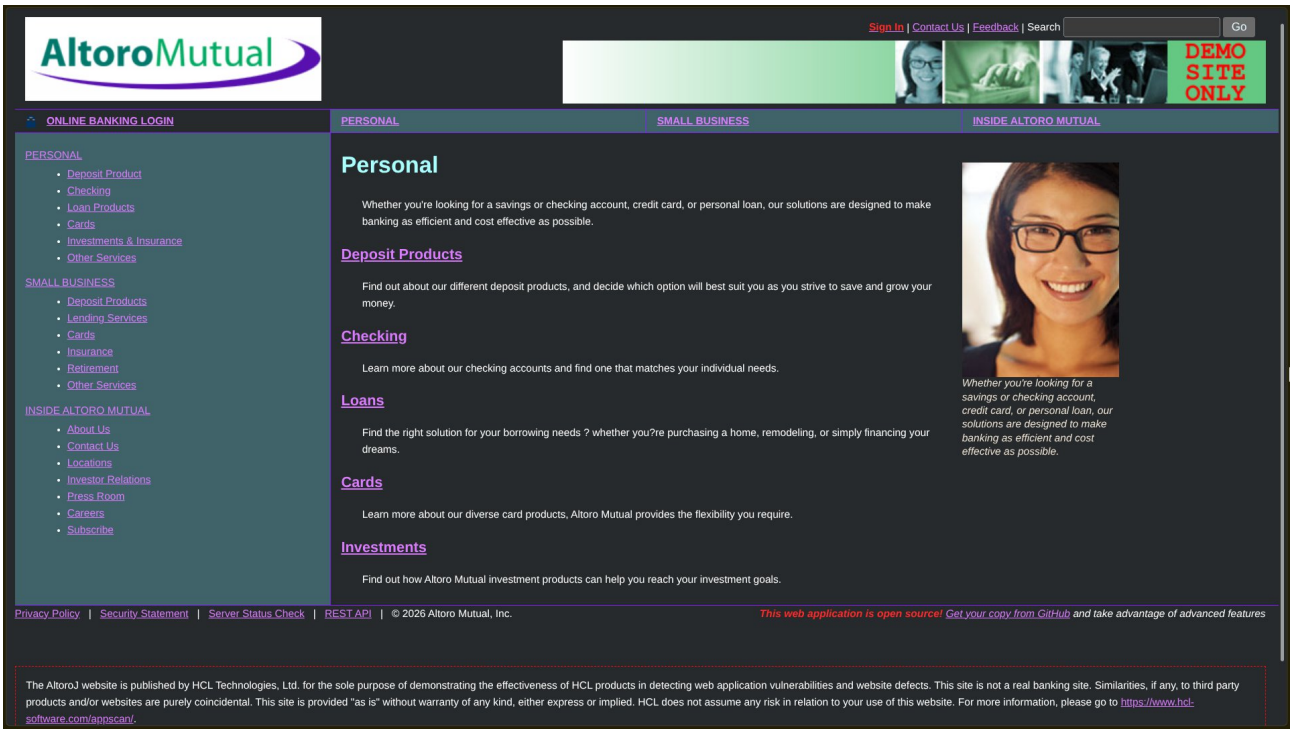


Figure 1. Personal Banking page — overview of individual customer services.

SCREENSHOT 02

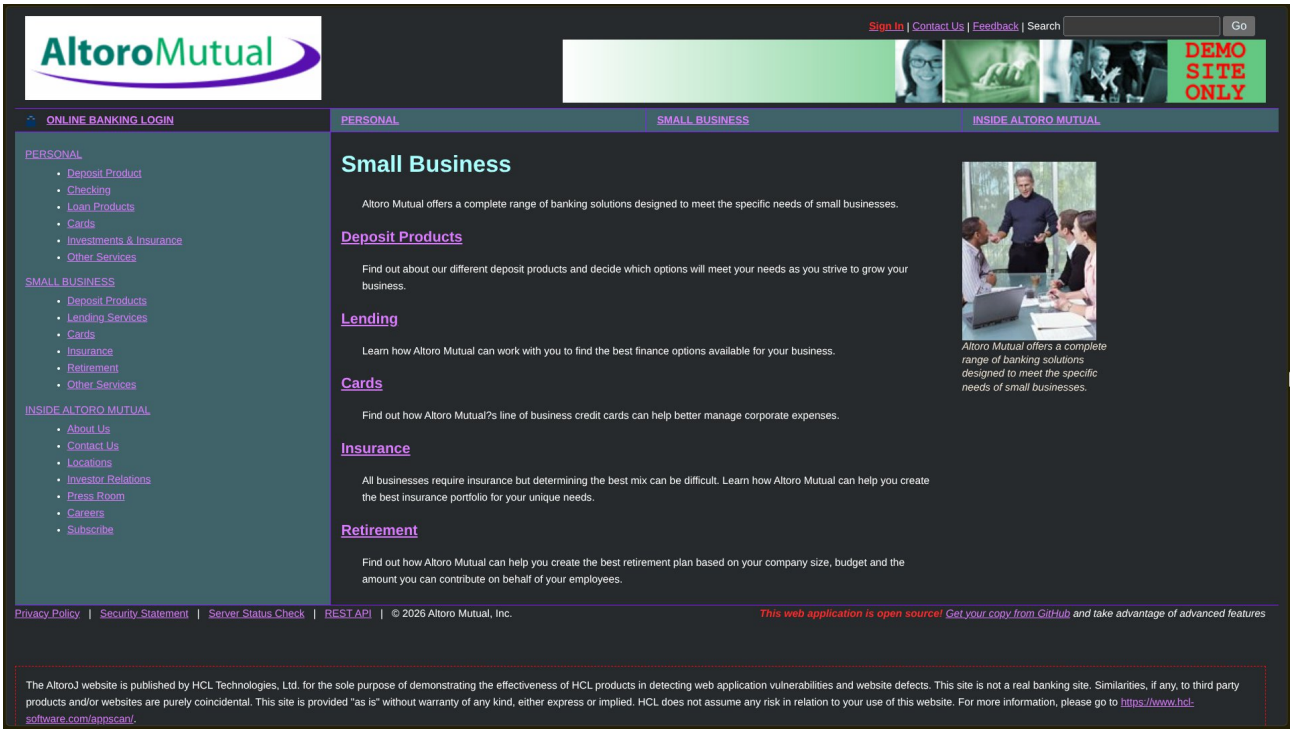


Figure 2. Small Business page — overview of business banking information.

SCREENSHOT 03

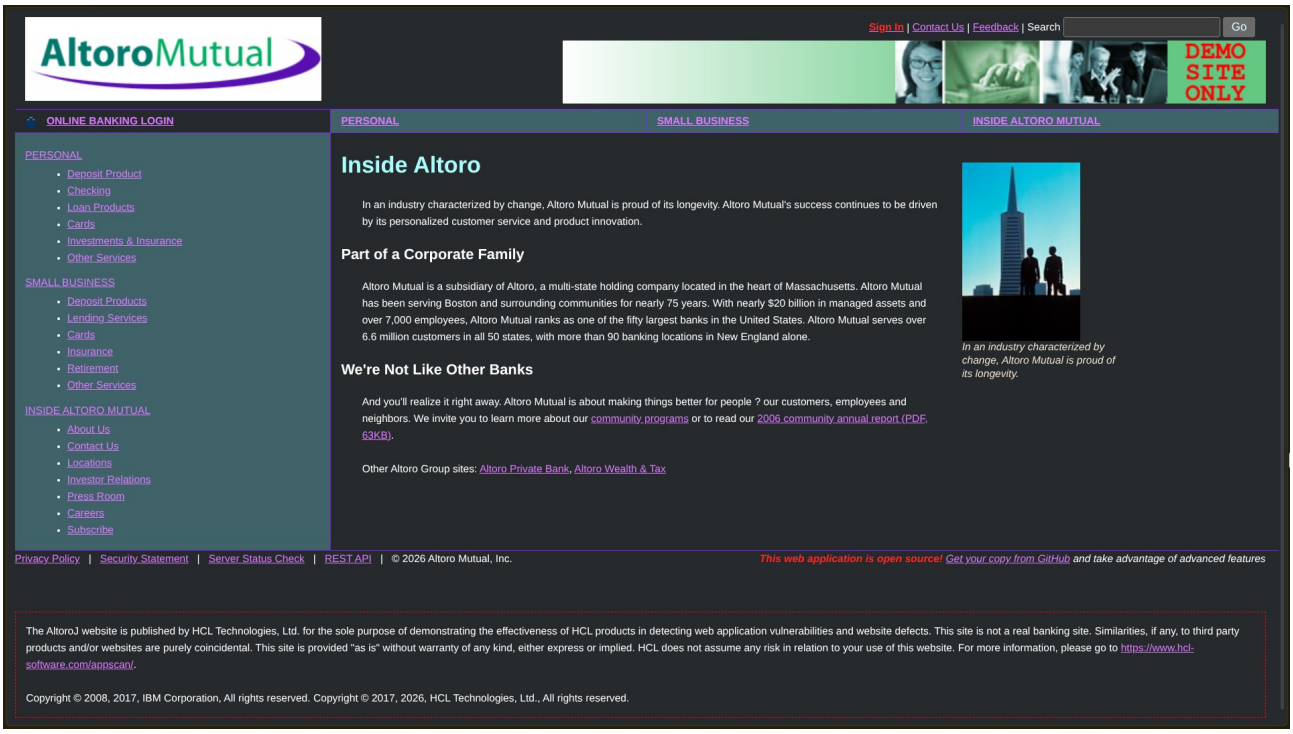


Figure 3. Inside Altoro Mutual page — general information section.

SCREENSHOT 04

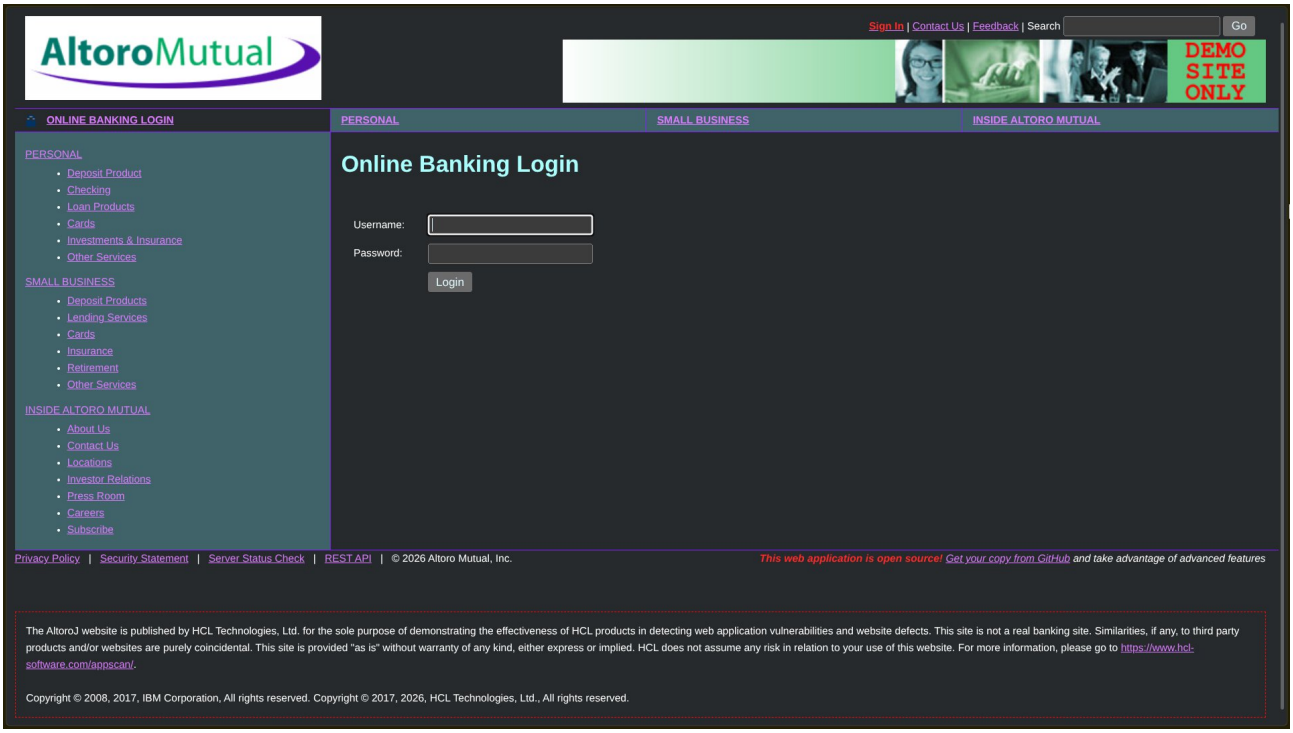


Figure 4. Online Banking login form — location of the authentication fields.

SCREENSHOT 05

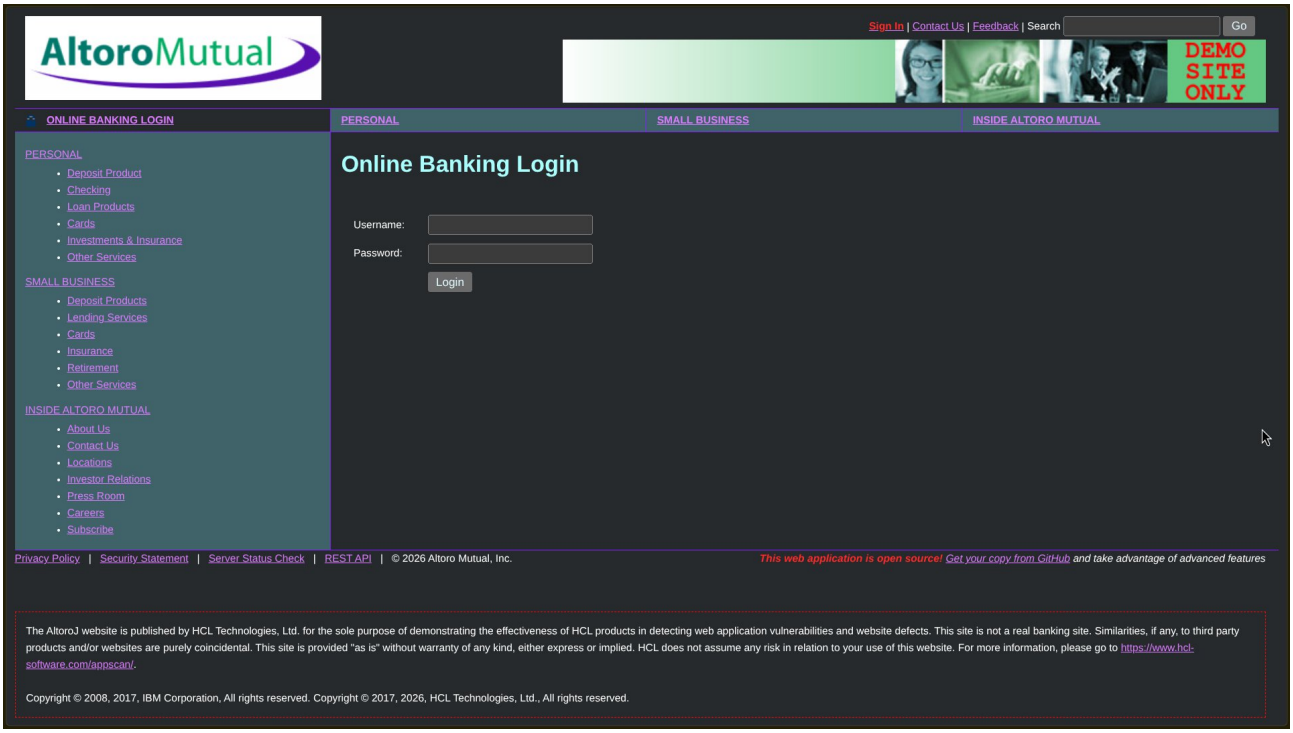


Figure 5. Login form view used during the recorded SQL injection checks.

SCREENSHOT 06



Figure 6. Search field visible in the site's top-right navigation area.

SCREENSHOT 07

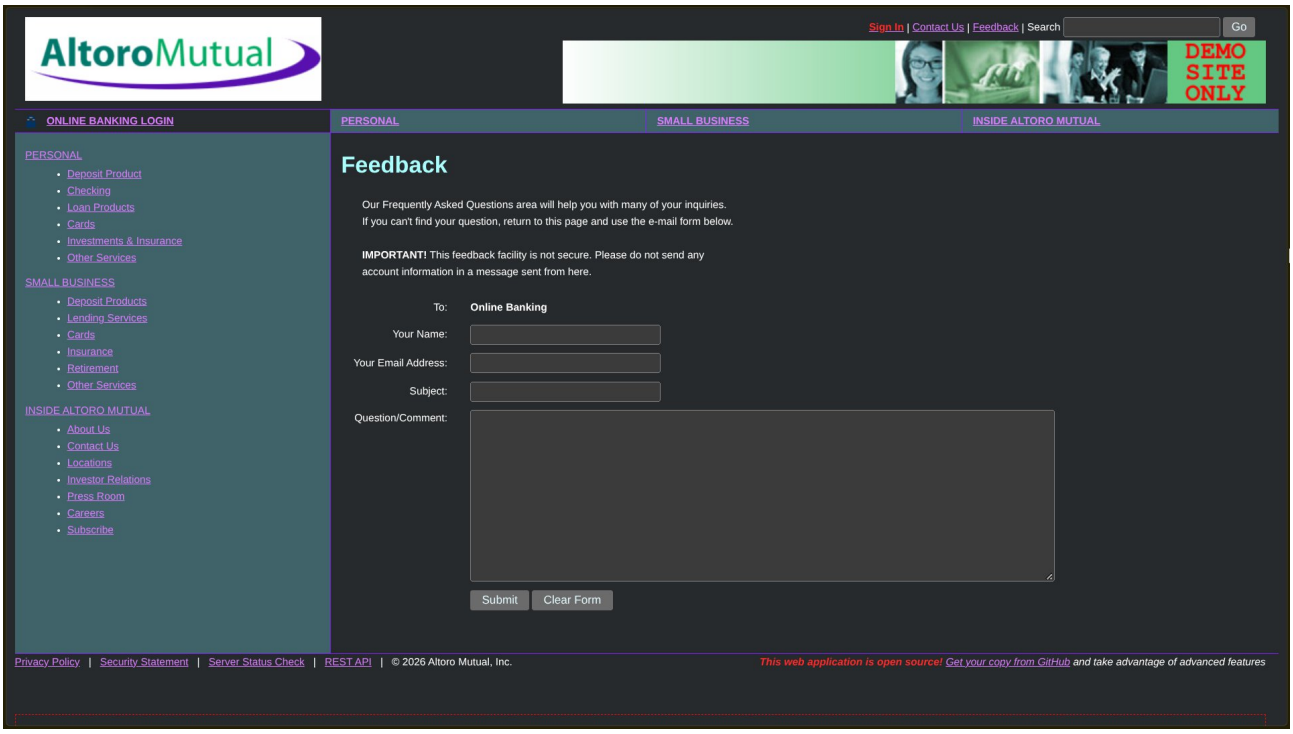


Figure 7. Feedback form — user-submitted comment fields.

SCREENSHOT 08

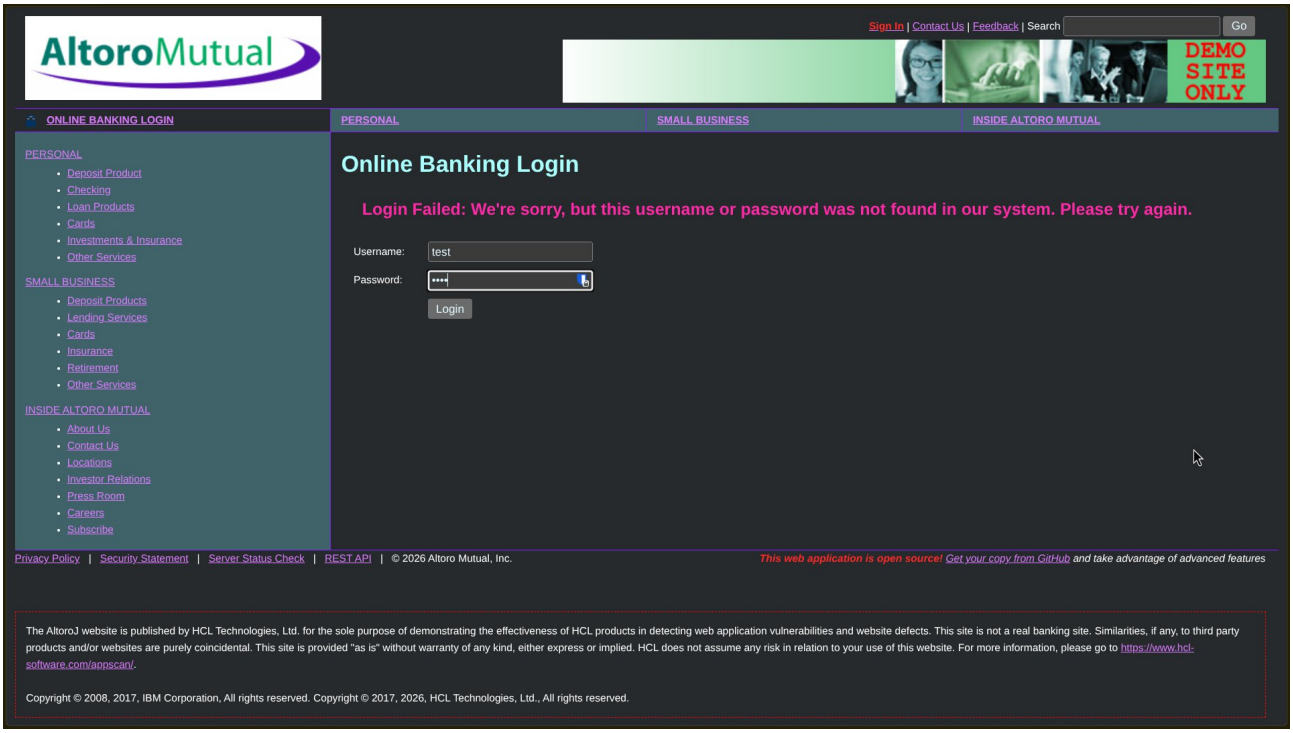


Figure 8. Baseline login attempt using the made-up credentials test/test.

SCREENSHOT 09

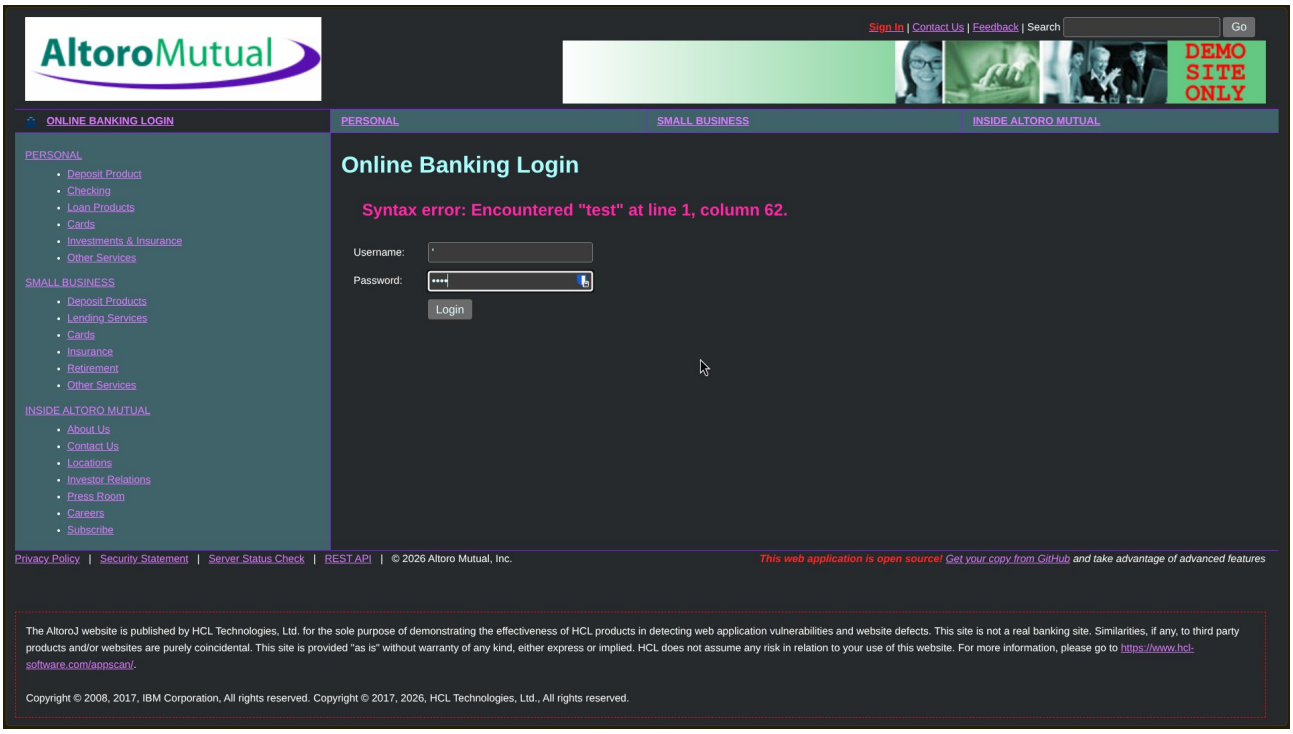


Figure 9. Username apostrophe probe — syntax-error response.

SCREENSHOT 10

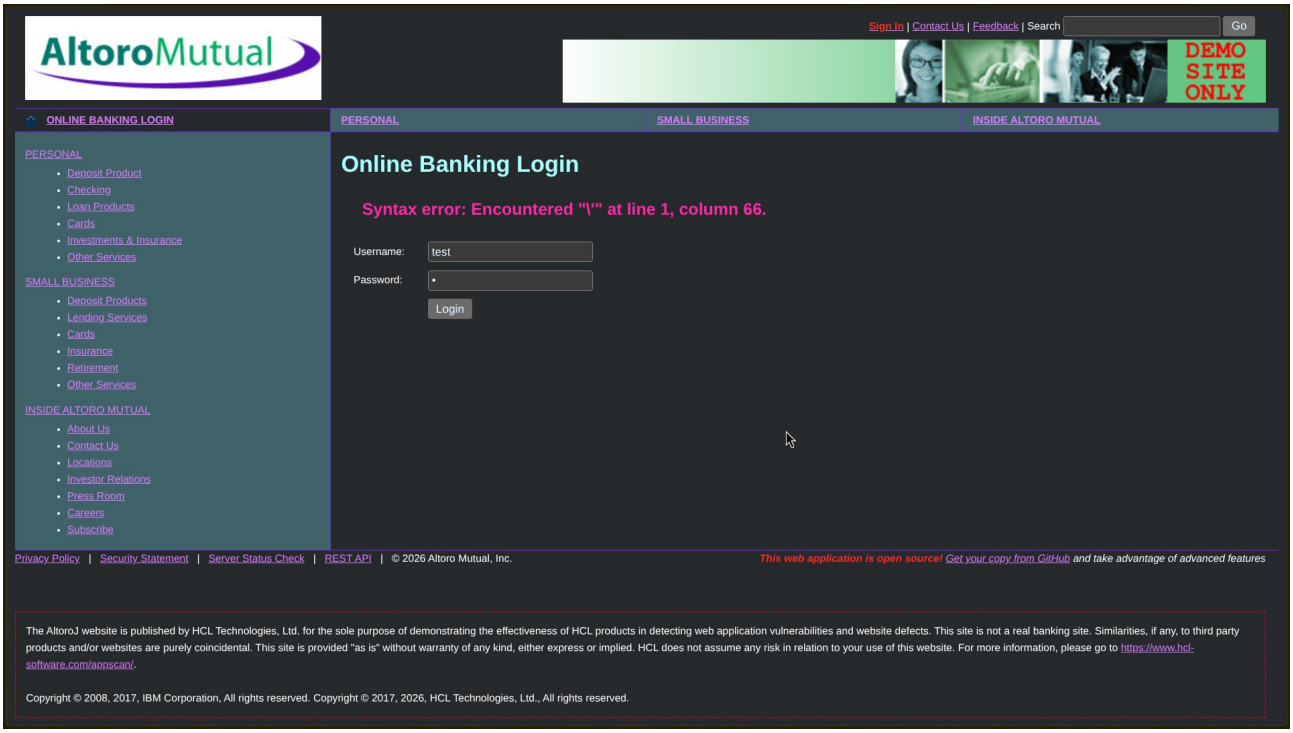


Figure 10. Password apostrophe probe — syntax-error response.

SCREENSHOT 11

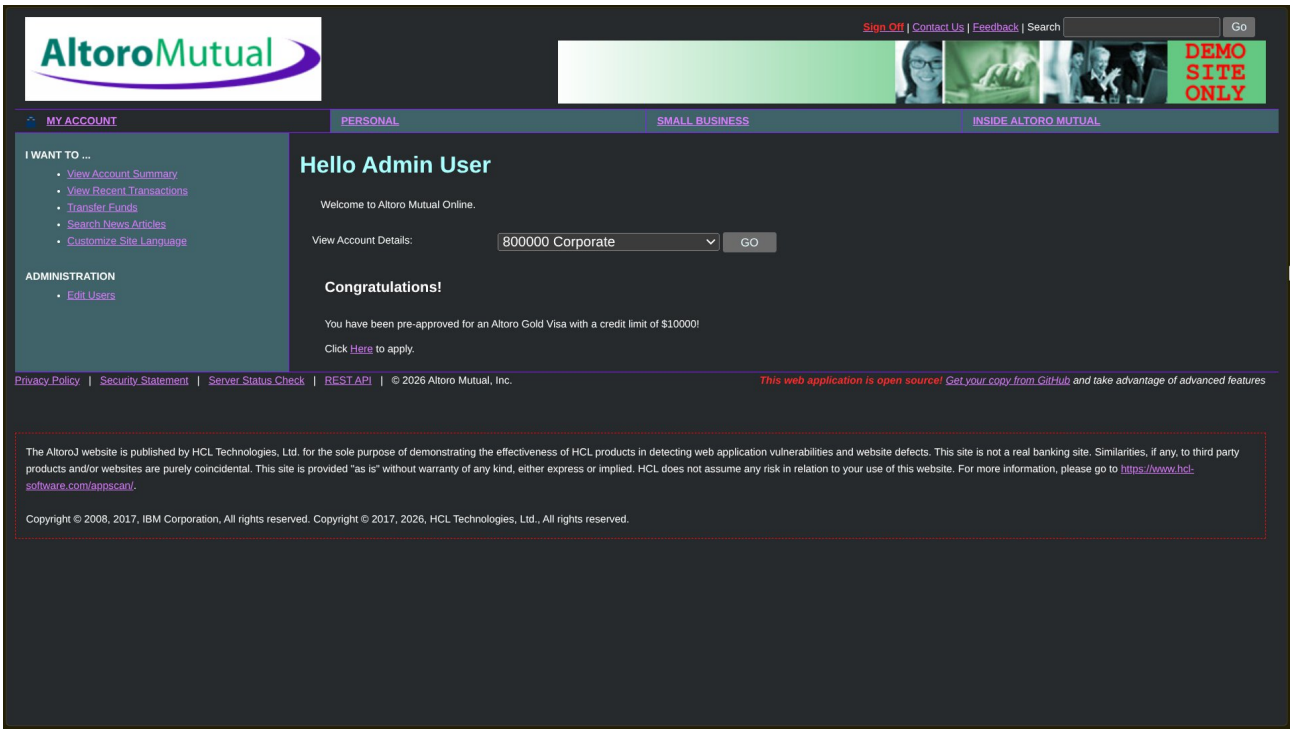


Figure 11. Optional SQL injection test — authenticated-looking welcome page.

SCREENSHOT 12

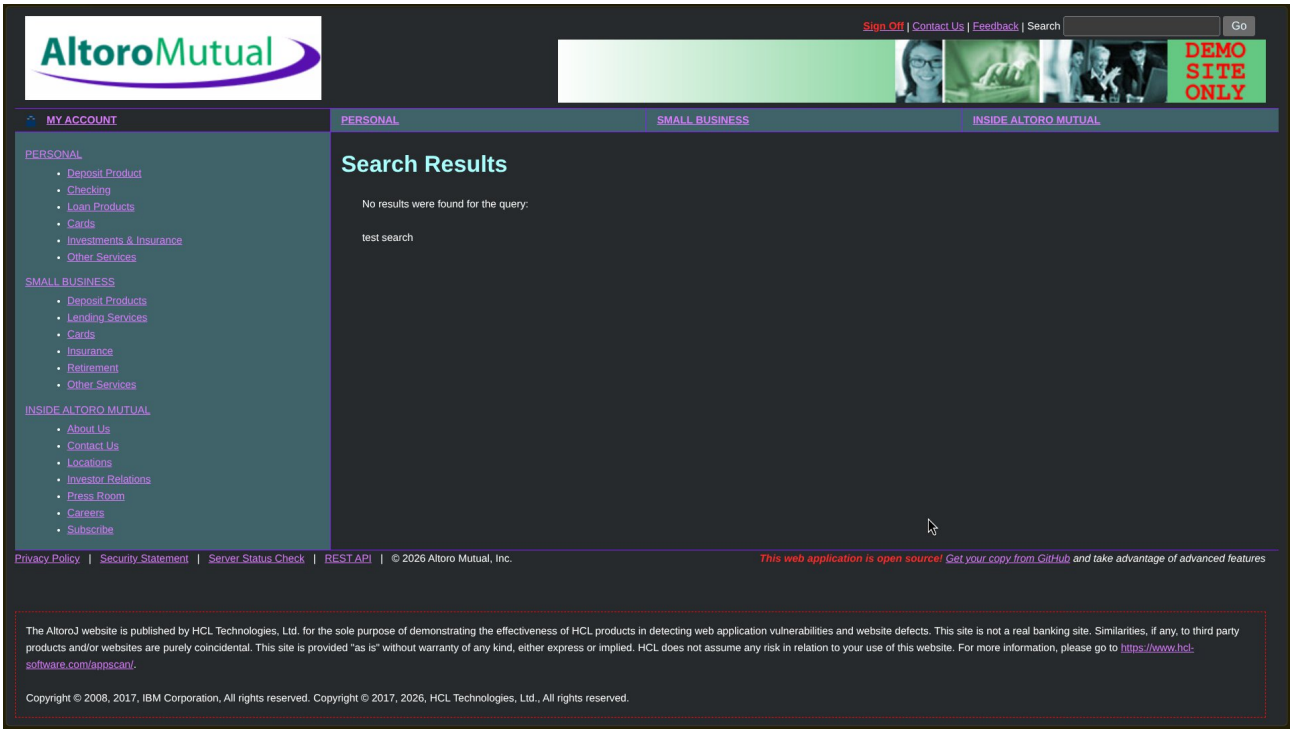


Figure 12. Search baseline — response to the ordinary query test search.

SCREENSHOT 13

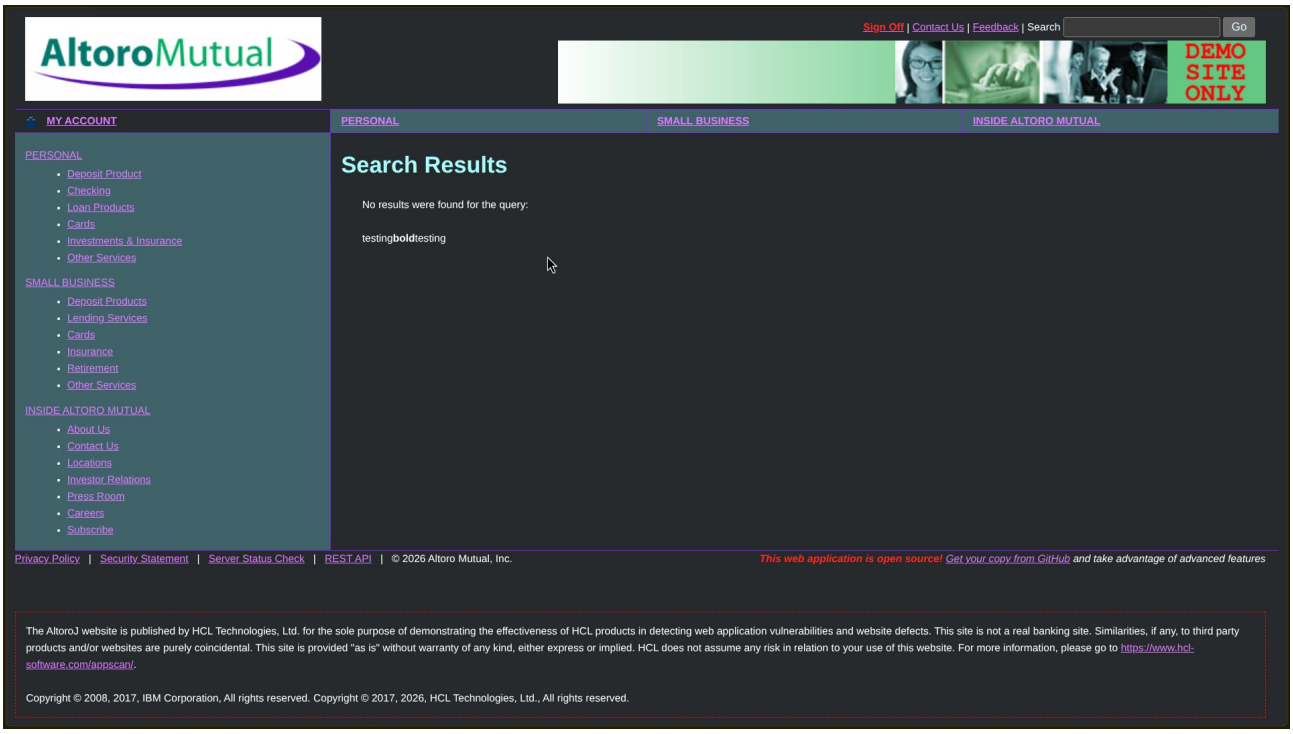


Figure 13. HTML-marker test — response to testingboldtesting.

SCREENSHOT 14

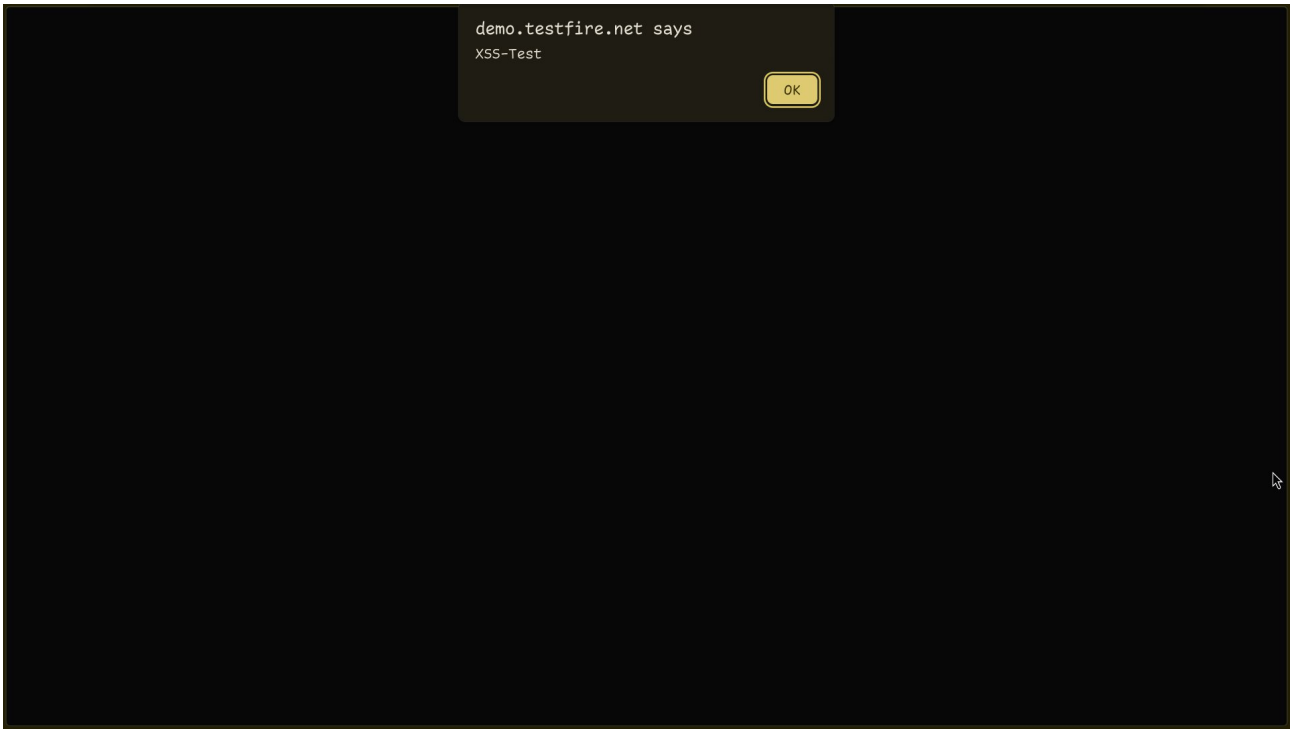


Figure 14. Script test — alert dialog observed after submitting the XSS test string.

6. Conclusion

The assessment demonstrated why login and search features require careful security testing. The recorded SQL syntax errors and apparent login bypass indicate SQL injection in the tested login flow, while the alert dialog indicates reflected XSS in the Search response. The most important fixes are parameterized queries, safe output encoding, generic error handling, and layered server-side controls. These conclusions are limited to the observed behavior of the authorized training application; they should not be generalized to other systems without testing.
